

Calories Burnt prediction using Machine Learning.

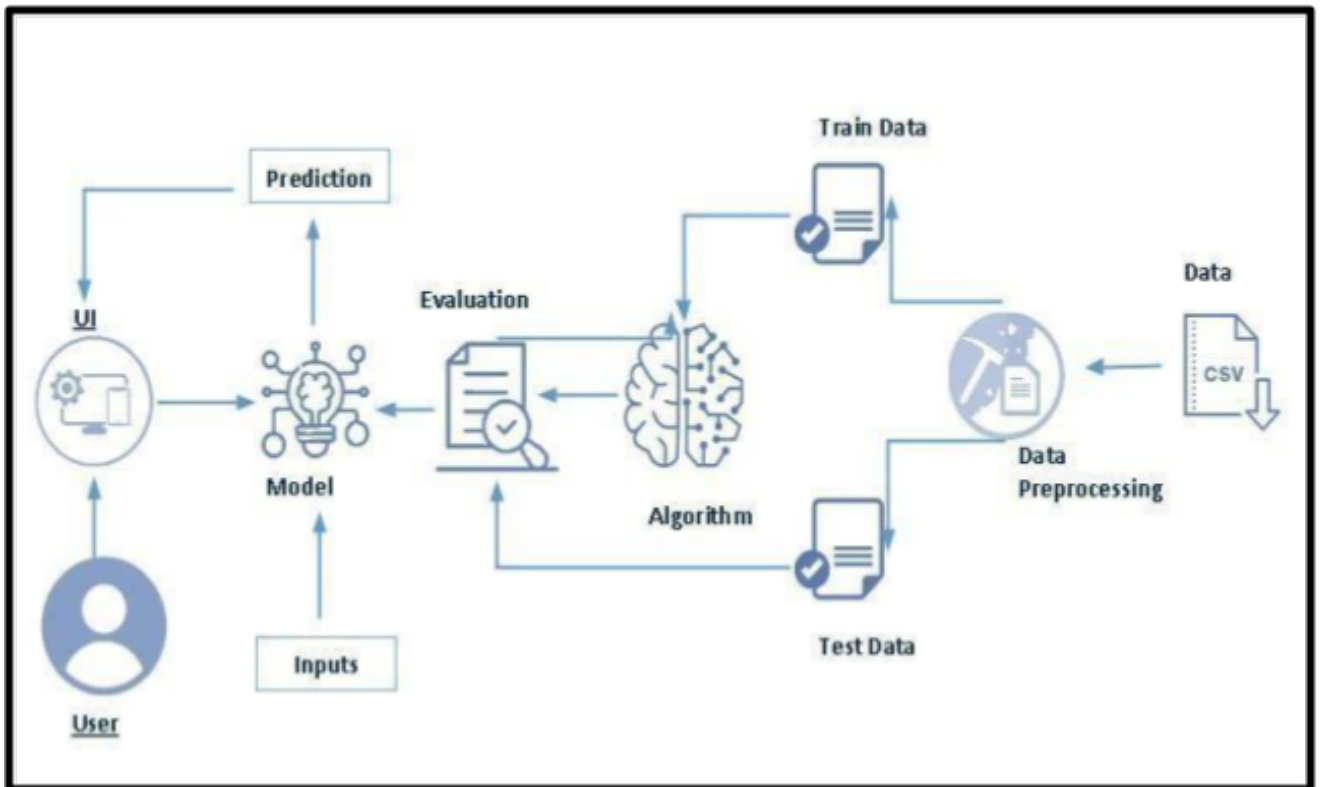
The objective of this project is to develop an end-to-end machine learning solution for predicting the number of calories burnt. The proposed solution involves the use of Machine learning algorithms to extract relevant features from the input and predict the calories burnt.

Creating an end-to-end machine learning project to predict calories burnt can offer a range of benefits, both from a health perspective and as a showcase of machine learning capabilities. Here are some of the key advantages:

1. **Personalized Fitness Guidance:** By predicting calorie burnt during different activities, machine Learning models can provide personalized fitness recommendations. Users can receive tailored exercise plans and activity suggestions based on their goals, fitness level, and preferences.
2. **Optimized Workouts:** Calorie prediction can help individuals optimize their workouts by selecting activities that align with their desired calorie burn. This can lead to more efficient and effective exercise routines, preventing wasted time on activities that might not align with their goals.
3. **Weight Management:** For individuals looking to manage their weight, understanding their predicted calorie expenditure can be crucial. It allows them to balance their calorie intake with their calorie burn, making informed decisions about their diet and exercise regimen.
4. **Motivation and Accountability:** Tracking calories burned can motivate individuals to stay active and achieve their fitness goals. Machine learning-powered apps can provide real-time feedback and progress updates, which can boost motivation and hold users accountable.

Let us look at the Technical Architecture of the project.

Technical Architecture:



Project Flow:

- The user interacts with the UI to enter the data.
- Uploaded input is analyzed by the model which is integrated/developed by you.
- Once the model analyzes the input the prediction is showcased on the

UI. To accomplish this, we have to complete all the activities listed below,

- Define Problem / Problem Understanding
 - 0 Specify the business problem
 - o Business requirements
 - o Literature Survey.
 - o Social or Business Impact.
- Data Collection & Preparation
 - o Collect the dataset
 - o Data Preparation
- Exploratory Data Analysis
 - 0 Descriptive statistical
 - o Visual Analysis
- Model Building
 - o Building a model.
 - 0 Training the model.
 - o Testing the model
- Model Deployment
 - 0 Save the best model
 - o Integrate with Web Framework
- Project Demonstration & Documentation
 - 0 Record explanation Video for project end to end solution

- Project Documentation-Step by step project development procedure

Prior Knowledge:

To complete this project, you must require following software's , concepts and packages

- VS Code :
 - Refer to the link below to download VS Code.
 - Link : <https://code.visualstudio.com/download>
- Create Project:
 - Open VS Code.
 - Create a Folder and name it "Calories_Burnt" .
- Machine Learning Concepts
 - Linear Regression: <https://towardsdatascience.com/linear-regression-detailed-view-ea73175f6e86>
 - XGBRegressor : <https://medium.com/@aryanbajaj13/prediction-using-xgb-regressor-using-python-3-ad1a57e105af>
 - Random Forest : <https://medium.com/towards-data-science/understanding-random-forest-58381e0602d2>
- Web concepts:
 - Get the gist on streamlit : <https://www.geeksforgeeks.org/a-beginners-guide-to-streamlit/>

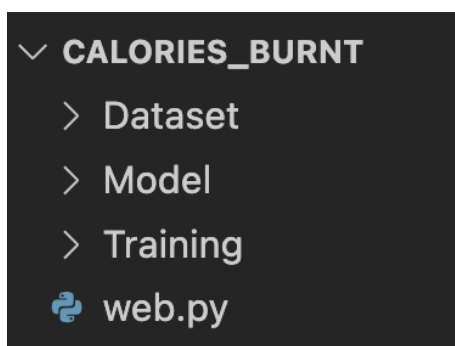
Project Objectives:

By the end of this project you will:

- Know fundamental concepts and techniques of Machine Learning.
- Know how to pre-process/clean the data using different data preprocessing techniques.
- Know how to build a web application using the Streamlit framework.

Project Structure:

Create the Project folder which contains files as shown below



- We are building a Streamlit application which needs a python script web.py for a website.
- Model folder contains your saved models.
- The Training folder contains your code for building/training/testing the model.
- Dataset folder contains your data.

Milestone 1: Define Problem / Problem Understanding

Activity 1: Specify the business problem

Refer Project Description

Activity 2: Business requirements

Here are some potential business requirements for a calorie burnt predictor using machine learning:

1. **Health Monitoring:** Calorie prediction can contribute to holistic health monitoring. It can be integrated into wearable devices, enabling users to track their daily activity levels and estimated energy expenditure. This information can be valuable for maintaining a healthy lifestyle.
2. **Research and Insights:** Aggregated and anonymized data from calorie burn predictions can be used for research purposes. It can contribute to studies about population-level activity trends, the effectiveness of different exercises, and the impact of physical activity on various health outcomes.
3. **Customized Nutrition Plans:** Integrating calorie burn predictions with nutritional information allows for more accurate customization of diet plans. Users can make better decisions about their food choices based on the energy they expend through different activities.

Activity 3: Literature Survey (Student Will Write)

A literature survey would involve researching and reviewing existing studies, articles, and other publications on the topic of project. The survey would aim to gather information on current systems, their strengths and weaknesses, and any gaps in knowledge that the project could address. The literature survey would also look at the methods and techniques used in previous projects, and any relevant data or findings that could inform the design and implementation of the current project.

Activity 4: Social or Business Impact.

1. **Health Equity:** Machine learning-powered fitness apps and wearables can make personalized health and fitness guidance more accessible to a wider range of individuals, regardless of their socioeconomic status. This can contribute to reducing health disparities by providing everyone with the tools to make healthier lifestyle choices.

2. **Market Differentiation:** Companies that offer accurate calorie burn prediction as part of their fitness apps or wearable devices can differentiate themselves in a competitive market. This unique feature can attract health-conscious consumers looking for personalized solutions.
3. **Subscription Models:** Accurate calorie burn prediction can be offered as a premium feature in subscription-based models, providing users with an incentive to upgrade and access more advanced fitness tracking capabilities.

Milestone 2: Data Collection & Preparation

ML depends heavily on data. It is the most crucial aspect that makes algorithm training possible. So, this section allows you to download the required dataset.

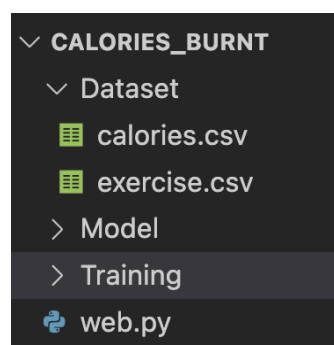
Activity 1: Collect the dataset.

There are many popular open sources for collecting the data. Eg: kaggle.com, UCI repository, etc.

This data is downloaded from kaggle.com. Please refer to the link given below to download the dataset.

[CLICK ME](#)

Make sure your directory looks like below



As the dataset is downloaded. Let us read and understand the data properly with the help of some visualization techniques and some analyzing techniques.

Note: There are a number of techniques for understanding the data. But here we have used some of it. In an additional way, you can use multiple techniques.

Under the Training folder create a Training.ipynb file in jupyter and start writing code in this file.

Activity 1.1: Importing the libraries

Import the necessary libraries as shown in the image.


```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from xgboost import XGBRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import LinearRegression
from sklearn import metrics
from sklearn.metrics import r2_score, mean_absolute_error
```

Activity 1.2: Data Loading and Preprocessing.

Reading calories.csv :

```
calories = pd.read_csv('../Dataset/calories.csv')
```

calories

	User_ID	Calories
0	14733363	231.0
1	14861698	66.0
2	11179863	26.0
3	16180408	71.0
4	17771927	35.0
...	...	...
14995	15644082	45.0
14996	17212577	23.0
14997	17271188	75.0
14998	18643037	11.0
14999	11751526	98.0

Reading exercise.csv :

```
exercise_data = pd.read_csv('../Dataset/exercise.csv')
```

exercise_data

	User_ID	Gender	Age	Height	Weight	Duration	Heart_Rate	Body_Temp
0	14733363	male	68	190.0	94.0	29.0	105.0	40.8
1	14861698	female	20	166.0	60.0	14.0	94.0	40.3
2	11179863	male	69	179.0	79.0	5.0	88.0	38.7
3	16180408	female	34	179.0	71.0	13.0	100.0	40.5
4	17771927	female	27	154.0	58.0	10.0	81.0	39.8
...	...	...	...	...	...	...	...	...
14995	15644082	female	20	193.0	86.0	11.0	92.0	40.4
14996	17212577	female	27	165.0	65.0	6.0	85.0	39.2
14997	17271188	female	43	159.0	58.0	16.0	90.0	40.1
14998	18643037	male	78	193.0	97.0	2.0	84.0	38.3
14999	11751526	male	63	173.0	79.0	18.0	92.0	40.5

15000 rows × 8 columns

Milestone 3: Exploratory Data Analysis

Activity 1: Combining Data Frames

```
calories_data = pd.concat([exercise_data, calories['Calories']], axis=1)
```

calories_data

	User_ID	Gender	Age	Height	Weight	Duration	Heart_Rate	Body_Temp	Calories
0	14733363	male	68	190.0	94.0	29.0	105.0	40.8	231.0
1	14861698	female	20	166.0	60.0	14.0	94.0	40.3	66.0
2	11179863	male	69	179.0	79.0	5.0	88.0	38.7	26.0
3	16180408	female	34	179.0	71.0	13.0	100.0	40.5	71.0
4	17771927	female	27	154.0	58.0	10.0	81.0	39.8	35.0
...	...	...	...	...	...	...	...	...	...
14995	15644082	female	20	193.0	86.0	11.0	92.0	40.4	45.0
14996	17212577	female	27	165.0	65.0	6.0	85.0	39.2	23.0
14997	17271188	female	43	159.0	58.0	16.0	90.0	40.1	75.0
14998	18643037	male	78	193.0	97.0	2.0	84.0	38.3	11.0
14999	11751526	male	63	173.0	79.0	18.0	92.0	40.5	98.0

15000 rows x 9 columns

```
calories_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15000 entries, 0 to 14999
Data columns (total 9 columns):
#   Column      Non-Null Count  Dtype
---  -
0   User_ID     15000 non-null  int64
1   Gender      15000 non-null  object
2   Age         15000 non-null  int64
3   Height      15000 non-null  float64
4   Weight      15000 non-null  float64
5   Duration    15000 non-null  float64
6   Heart_Rate  15000 non-null  float64
7   Body_Temp   15000 non-null  float64
8   Calories    15000 non-null  float64
dtypes: float64(6), int64(2), object(1)
memory usage: 1.0+ MB
```

Checking for null values :

```
calories_data.isnull().sum()
```

```
User_ID      0
Gender       0
Age          0
Height       0
Weight       0
Duration     0
Heart_Rate   0
Body_Temp    0
Calories     0
dtype: int64
```

Description of the dataset :

```
calories_data.describe()
```

	User_ID	Age	Height	Weight	Duration	Heart_Rate	Body_Temp	Calories
count	1.500000e+04	15000.000000	15000.000000	15000.000000	15000.000000	15000.000000	15000.000000	15000.000000
mean	1.497736e+07	42.789800	174.465133	74.966867	15.530600	95.518533	40.025453	89.539533
std	2.872851e+06	16.980264	14.258114	15.035657	8.319203	9.583328	0.779230	62.456978
min	1.000116e+07	20.000000	123.000000	36.000000	1.000000	67.000000	37.100000	1.000000
25%	1.247419e+07	28.000000	164.000000	63.000000	8.000000	88.000000	39.600000	35.000000
50%	1.499728e+07	39.000000	175.000000	74.000000	16.000000	96.000000	40.200000	79.000000
75%	1.744928e+07	56.000000	185.000000	87.000000	23.000000	103.000000	40.600000	138.000000
max	1.999965e+07	79.000000	222.000000	132.000000	30.000000	128.000000	41.500000	314.000000

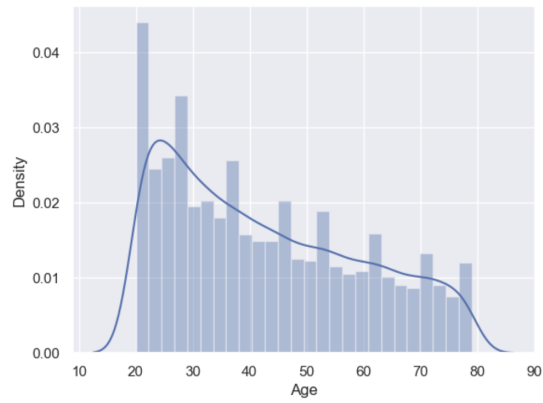
Activity 2: Visualization

- Setting up the sns plot :

```
sns.set()
```

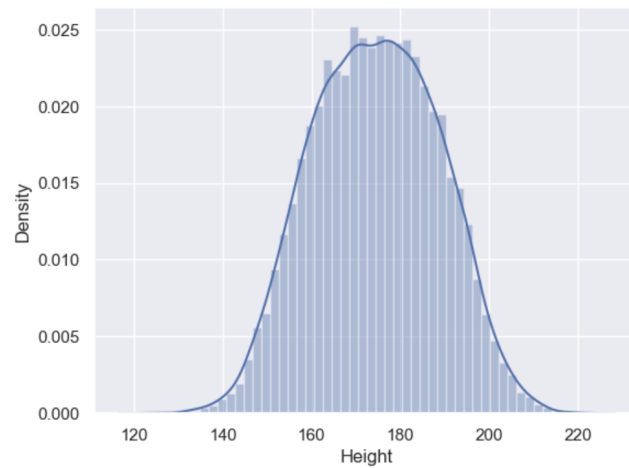
- Visualizing the age column.

```
# finding the distribution of "Age" column
sns.distplot(calories_data['Age'])
```



- Visualizing Height column.

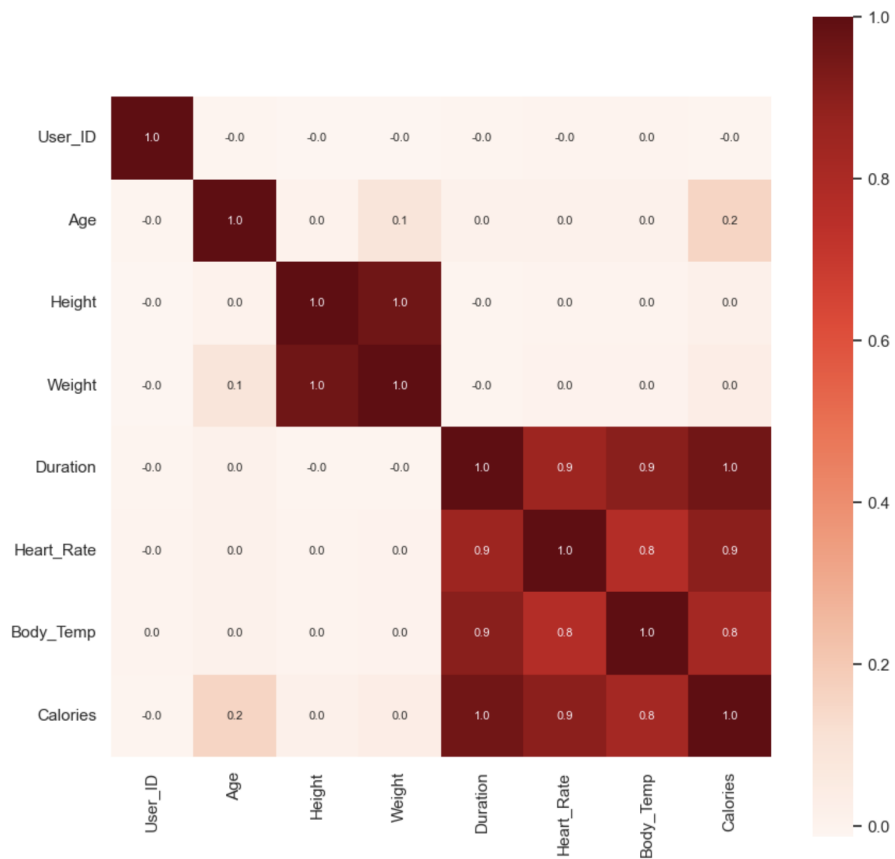
```
# finding the distribution of "Height" column
sns.distplot(calories_data['Height'])
```



- Let's find the correlation between features.

```
correlation = calories_data.corr()
```

```
plt.figure(figsize=(10,10))
sns.heatmap(correlation, cbar=True, square=True, fmt='.1f', annot=True, annot_kws={'size':8}, cmap='Reds')
```



- Converting gender column to integer.

```
calories_data.replace({"Gender":{"male":0,'female':1}}, inplace=True)
```

calories_data

	User_ID	Gender	Age	Height	Weight	Duration	Heart_Rate	Body_Temp	Calories
0	14733363	0	68	190.0	94.0	29.0	105.0	40.8	231.0
1	14861698	1	20	166.0	60.0	14.0	94.0	40.3	66.0
2	11179863	0	69	179.0	79.0	5.0	88.0	38.7	26.0
3	16180408	1	34	179.0	71.0	13.0	100.0	40.5	71.0
4	17771927	1	27	154.0	58.0	10.0	81.0	39.8	35.0
...	...	...	...	...	...	...	...	...	...
14995	15644082	1	20	193.0	86.0	11.0	92.0	40.4	45.0
14996	17212577	1	27	165.0	65.0	6.0	85.0	39.2	23.0
14997	17271188	1	43	159.0	58.0	16.0	90.0	40.1	75.0
14998	18643037	0	78	193.0	97.0	2.0	84.0	38.3	11.0
14999	11751526	0	63	173.0	79.0	18.0	92.0	40.5	98.0

15000 rows x 9 columns

Activity 3: Splitting data into train and test sets

As we've seen there is no correlation between weight and calories burnt. So, we'll drop the weight column.

We are gonna drop the column while splitting the data into train and test datasets.

```
X = calories_data.drop(columns=['User_ID', 'Calories', 'Weight'], axis=1)
Y = calories_data['Calories']
```

X

	Gender	Age	Height	Duration	Heart_Rate	Body_Temp
0	0	68	190.0	29.0	105.0	40.8
1	1	20	166.0	14.0	94.0	40.3
2	0	69	179.0	5.0	88.0	38.7
3	1	34	179.0	13.0	100.0	40.5
4	1	27	154.0	10.0	81.0	39.8
...	...	...	...	...	...	...
14995	1	20	193.0	11.0	92.0	40.4
14996	1	27	165.0	6.0	85.0	39.2
14997	1	43	159.0	16.0	90.0	40.1
14998	0	78	193.0	2.0	84.0	38.3
14999	0	63	173.0	18.0	92.0	40.5

15000 rows × 6 columns

Y

```
0      231.0
1       66.0
2       26.0
3       71.0
4       35.0
```

...

```
14995    45.0
14996    23.0
14997    75.0
14998    11.0
14999    98.0
```

Name: Calories, Length: 15000, dtype: float64

```
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=2)
```

```
print(X.shape, X_train.shape, X_test.shape)
```

```
(15000, 6) (12000, 6) (3000, 6)
```

Milestone 4: Model Training

In this step we'll select models and train them, step by step.

We'll use 3 machine learning algos and find the best one out.

Model 1: Linear Regression

Loading model :

```
model = LinearRegression()
```

Training and calculating the accuracy

```
model.fit(X_train, Y_train)
```

```
LinearRegression()
```

```
test_data_prediction = model.predict(X_test)
```

```
print(test_data_prediction)
```

```
[136.13750038 181.1106889  51.22849358 ... 157.10316595  17.48925282
 101.38944265]
```

```
mae = metrics.mean_absolute_error(Y_test, test_data_prediction)
```

```
print("Mean Absolute Error = ", mae)
```

```
Mean Absolute Error =  8.398683417976256
```

```
print(r2_score(Y_test, test_data_prediction))
```

```
0.9667019173772892
```

Accuracy is near 96%.

Mean Absolute Error is near 8.3.

Model 2: Random Forest Regressor

Loading model :

```
model = RandomForestRegressor()
```

Training and calculating the accuracy

```
model.fit(X_train, Y_train)
```

```
RandomForestRegressor()
```

```
test_data_prediction = model.predict(X_test)
```

```
mae = metrics.mean_absolute_error(Y_test, test_data_prediction)
```

```
print("Mean Absolute Error = ", mae)
```

```
Mean Absolute Error = 1.8564874444444444
```

```
print(r2_score(Y_test, test_data_prediction))
```

```
0.997957059305419
```

Accuracy is somewhere around 99%

Mean Absolute Error is near 1.85 .

Model 3: XGBRegressor

Loading Model :

```
model = XGBRegressor()
```

Training and calculating the accuracy

```
model.fit(X_train, Y_train)
```

```
XGBRegressor(base_score=None, booster=None, callbacks=None,  
             colsample_bylevel=None, colsample_bynode=None,  
             colsample_bytree=None, early_stopping_rounds=None,  
             enable_categorical=False, eval_metric=None, feature_types=None,  
             gamma=None, gpu_id=None, grow_policy=None, importance_type=None,  
             interaction_constraints=None, learning_rate=None, max_bin=None,  
             max_cat_threshold=None, max_cat_to_onehot=None,  
             max_delta_step=None, max_depth=None, max_leaves=None,  
             min_child_weight=None, missing=None, monotone_constraints=None,  
             n_estimators=100, n_jobs=None, num_parallel_tree=None,  
             predictor=None, random_state=None, ...)
```

```
test_data_prediction = model.predict(X_test)
```

```
mae = metrics.mean_absolute_error(Y_test, test_data_prediction)
```

```
print("Mean Absolute Error = ", mae)
```

```
Mean Absolute Error = 1.682497463936607
```

```
print(r2_score(Y_test, test_data_prediction))
```

```
0.9985176828000004
```

Accuracy is somewhere around 99%.

Mean Absolute Error is 1.68.

Milestone 5: Model Deployment

Activity 1: Save the best model

After analyzing the accuracy and mean absolute error of above models we'll select the best model out of those 3 models.

As we can see XGBRegressor is performing well out of all the models.

```
import pickle
```

```
pickle.dump(model, open('../Model/model.pkl', 'wb'))
```

Activity 2: Integrate with Web Framework

In this section, we will be building a web application that is integrated to the model we built.

We will be using the streamlit package for our website development.

Streamlit is a free and open-source framework to rapidly build and share beautiful machine learning and data science web apps.

Activity 2.1: Create a web.py file and import necessary packages:

✓ CALORIES_BURNT

> Dataset

> Model

> Training

🔗 web.py

```
import streamlit as st
import pickle
import numpy as np
```

NOTE : If you get any error like “streamlit not found”. You have to make sure streamlit is installed on your system.

- Running the command “pip install streamlit” would do that job for you.

Activity 2.2: Model loading:

```
model = pickle.load(open('./Model/model.pkl','rb'))

gender = [0,1]
```

We have also set the values for gender.

Activity 2.3: Accepting the input from user and prediction

```
st.title('Number of Calories Burnt Predictor')
Gender = st.selectbox('Select your gender (0-male, 1-female)',sorted(gender))
```

```
col1, col2 = st.columns(2)
with col1:
    Age = st.number_input('Age')
with col2:
    Height = st.number_input('Height')

col3,col4,col5 = st.columns(3)

with col3:
    Duration = st.number_input('Exercise Duration (in mins)')
with col4:
    Heart_Rate = st.number_input('Heart Rate')
with col5:
    Body_Temp = st.number_input('Body Temp (in Celsius)')

if st.button('Predict Calories Burnt'):

    (variable) input_df: ndarray[Any, dtype[generic]] Body_Temp]
    input_df = input_df.reshape(1,-1)
    result = model.predict(input_df)
    st.header("Calories burnt - " + str(int(result[0])))
```

To run your website go to your terminal with your respective directory and run the command :

- “streamlit run web.py”

You can now view your Streamlit app in your browser.

Local URL: <http://localhost:8501>

Network URL: <http://192.168.1.11:8501>

For better performance, install the Watchdog module:

```
$ xcode-select --install
```

```
$ pip install watchdog
```

On successful execution you'll get to see this in your terminal.

HOW DOES YOUR WEBSITE LOOK LIKE ?

- Before entering the data :

The screenshot shows a web application titled "Number of Calories Burnt Predictor" on a dark background. The interface includes several input fields and a prediction button. At the top, there is a dropdown menu for "Select your gender (0-male, 1-female)" with the value "0" selected. Below this are two input fields for "Age" and "Height", both set to "0.00". Further down are three input fields for "Exercise Duration (in mins)", "Heart Rate", and "Body Temp (in Celsius)", all set to "0.00". Each of these three input fields has minus and plus buttons on its right side. At the bottom left, there is a button labeled "Predict Calories Burnt".

- After entering the data :\

Number of Calories Burnt Predictor

Select your gender (0-male, 1-female)

0

Age

23.00 - +

Height

172.00 - +

Exercise Duration (in mins)

60.00 - +

Heart Rate

140.00 - +

Body Temp (in Celsius)

38.00 - +

Predict Calories Burnt

Calories burnt - 248

Milestone 6: Project Demonstration & Documentation

Below mentioned deliverables to be submitted along with other deliverables.

Activity 1: - Record explanation Video for project end to end solution.

Activity 2: - Project Documentation-Step by step project development procedure.

Create a document as per the template provided.